# Chapter 27 | Exercise solutions with Python

*Time-Series Forecasting*

This notebook solves Exercises 5–8. It checks which observations belong to a rolling window, follows seasonal predictions across cycles, and separates point error from interval coverage and width.

The examples run locally with NumPy. The hand calculations give expected values for the windows and metrics before the notebook compares them with the chapter functions. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch27_time_series"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

## 5. Inspect rolling windows

Imagine pausing the series just before the observation at index 3. The three values already available are $(0, 1, 2)$. The next two values, $(3, 4)$, are the targets we will score against later. The **forecast origin** is the index of the first unknown value, so this window has origin 3. The past ends at index 2.

For an origin $t$, a lookback of $L$ uses indices $t - L$ through $t - 1$. A horizon of $H$ uses targets from $t$ through $t + H - 1$. In a series of length $N$ with indices 0 through $N - 1$, the first past index must satisfy $t - L \geq 0$, and the last target must satisfy $t + H - 1 \leq N - 1$. Together these give $L \leq t \leq N - H$. Here $L = 3$, $H = 2$, and $N = 10$, so the valid origins are 3 through 8.

```
from examples.ch27_time_series.model import rolling_windows

window_values = np.arange(10)
lookback, window_horizon = 3, 2
window_origins, past_rows, future_rows = rolling_windows(
    window_values, lookback, window_horizon
)

print("Origins", window_origins)
for row in (0, -1):
    print(
        f"Origin {window_origins[row]}  "
        f"past {past_rows[row]}  future {future_rows[row]}"
    )
```

```
OUTPUT
Origins [3 4 5 6 7 8]
Origin 3  past [0. 1. 2.]  future [3. 4.]
Origin 8  past [5. 6. 7.]  future [8. 9.]
```

At origin 8 the past is $(5, 6, 7)$ and the targets are $(8, 9)$. The last target is $8 + 2 - 1 = 9$, the final index in the input. Origin 9 would need targets at indices 9 and 10, so it would have only one observed target. The rows here show values, which happen to equal their indices because the series is 0 through 9.

To check every row independently, slide a block of five values through the input. Split each block after its first three values. For example, the block $(1, 2, 3, 4, 5)$ gives past $(1, 2, 3)$ and targets $(4, 5)$ at origin 4.

```
expected_origins = [3, 4, 5, 6, 7, 8]
blocks = [window_values[start:start + 5] for start in range(6)]
expected_past = np.stack([block[:3] for block in blocks])
expected_future = np.stack([block[3:] for block in blocks])

np.testing.assert_array_equal(window_origins, expected_origins)
np.testing.assert_array_equal(past_rows, expected_past)
np.testing.assert_array_equal(future_rows, expected_future)
print("Checked the origins, past values, and targets of all six windows.")
```

```
OUTPUT
Checked the origins, past values, and targets of all six windows.
```

## 6. Forecast beyond a season

The history $(2, 5, 3, 6)$ contains two seasons of two values each. A season is one full repetition of the pattern. The seasonal-naive forecast copies the latest season, $(3, 6)$. It repeats that pair until it has five predictions, giving $(3, 6, 3, 6, 3)$.

Drift is the estimated change from one season to the next. To find it, compare each position in the second season with the same position in the first. The differences are $3 - 2 = 1$ and $6 - 5 = 1$, so their mean is $d = (1 + 1)/2 = 1$. This is one unit per **season**, which lasts two steps here.

The first two forecasts are one season ahead of their matching observations and receive $d$ each. Horizons 3 and 4 are two seasons ahead and receive $2d$ each. Horizon 5 receives $3d$.

```
from examples.ch27_time_series.model import (
    fit_seasonal_drift,
    seasonal_drift,
    seasonal_naive,
)

season_history = np.array([2, 5, 3, 6])
period, season_horizon = 2, 5
naive_forecast = seasonal_naive(season_history, period, season_horizon)
fitted_drift = fit_seasonal_drift(season_history, period)
drift_forecast = seasonal_drift(
    season_history, period, season_horizon, fitted_drift
)

print(f"Fitted drift {fitted_drift:g} unit per season")
print("Horizon   Seasonal naive   Added drift   With drift")
for step, (base, forecast) in enumerate(
    zip(naive_forecast, drift_forecast), start=1
):
    print(f"{step:>7} {base:>16g} {forecast-base:>13g} {forecast:>12g}")
```

```
OUTPUT
Fitted drift 1 unit per season
Horizon   Seasonal naive   Added drift   With drift
      1                3             1            4
      2                6             1            7
      3                3             2            5
      4                6             2            8
      5                3             3            6
```

**Horizon 3** is the first prediction to receive two drift increments. The first forecast cycle gives $(3 + 1, 6 + 1) = (4, 7)$. The third prediction returns to the first position of the season, giving $3 + 2(1) = 5$. The fourth gives $6 + 2(1) = 8$, and the fifth starts the third cycle with $3 + 3(1) = 6$.

For a separate check of the drift forecast, extend the history one step at a time. For each new value, take the value two steps back and add one unit. The first two new values use the observed pair $(3, 6)$. For horizon 3, the value two steps back is the first prediction, 4, so the new value is $4 + 1 = 5$. This step-by-step calculation checks all five predictions.

```
expected_cycle = [3, 6, 3, 6, 3]
first_change = season_history[2] - season_history[0]
second_change = season_history[3] - season_history[1]
expected_drift = (first_change + second_change) / 2

extended = season_history.tolist()
for _ in range(season_horizon):
    extended.append(extended[-period] + expected_drift)
expected_forecast = extended[len(season_history):]

np.testing.assert_array_equal(naive_forecast, expected_cycle)
assert fitted_drift == expected_drift == 1
np.testing.assert_array_equal(drift_forecast, expected_forecast)
np.testing.assert_array_equal(
    drift_forecast - naive_forecast, [1, 1, 2, 2, 3]
)
print("Checked the cycle, fitted drift, and all five forecasts.")
```

```
OUTPUT
Checked the cycle, fitted drift, and all five forecasts.
```

## 7. Change what the baseline learned

At origin 24, the shop has observed values through index 23. The chapter's seasonal-naive forecast copies the last observed values for the three matching seasonal positions. It predicts $(20, 24, 21)$, while the actual values are $(24, 28, 25)$. Each prediction misses by four units. A model that learned a drift of four would add four to each prediction.

Run the chapter's example as a module, as in the listing. The notebook uses the current Python interpreter and reads the JSON from a temporary directory. The mean absolute error (MAE) in each column averages the absolute errors at that horizon across all six test origins.

```
import json
import subprocess
from tempfile import TemporaryDirectory

with TemporaryDirectory() as temporary_directory:
    record_path = Path(temporary_directory) / "ch27.json"
    subprocess.run(
        [
            sys.executable,
            "-m", "examples.ch27_time_series.run_examples",
            "--output", str(record_path),
        ],
        cwd=repo_root,
        check=True,
        capture_output=True,
    )
    record = json.loads(record_path.read_text(encoding="utf-8"))

rolling = record["rolling"]
naive_mae = rolling["mae_seasonal_naive"]
fitted_mae = rolling["mae_seasonal_drift"]
assert rolling["fitted_drift"] == 4
assert naive_mae == [4, 4, 4]
assert fitted_mae == [0, 0, 0]

print("Fitted drift per season", rolling["fitted_drift"])
print("Horizons              1    2    3")
print("Seasonal-naive MAE  ", naive_mae)
print("Fitted-drift MAE    ", fitted_mae)
```

```
OUTPUT
Fitted drift per season 4.0
Horizons              1    2    3
Seasonal-naive MAE   [4.0, 4.0, 4.0]
Fitted-drift MAE     [0.0, 0.0, 0.0]
```

The fixture adds a repeating four-position pattern to the time index. Advancing four steps returns to the same pattern value and raises the series by four. This explains why the fitted drift is four and why it removes all the errors in this fixture. That exact correction depends on how this toy series was made.

Now replace the fitted drift with zero. Origin $o$ is the first unknown index, so `values[:origin]` ends at $o - 1$. At later origins, earlier test observations have arrived and can enter the history. The three values at indices $o$, $o + 1$, and $o + 2$ are still withheld for scoring.

With period four, the first three steps copy indices $o - 4$, $o - 3$, and $o - 2$. They are all in the first forecast season, so the drift model adds one increment to each. With a zero increment, those three predictions stay equal to the seasonal-naive predictions. Check both forecasts at every origin against the values at the three source indices.

```
from examples.ch27_time_series.model import seasonal_drift
from examples.ch27_time_series.model import seasonal_naive
from examples.ch27_time_series.run_examples import series_example

values = series_example()
test_origins = rolling["test_origins"]
period, horizon = 4, 3
assert test_origins == list(range(24, 30))
assert rolling["period"] == period and rolling["horizon"] == horizon

zero_drift_forecasts = []
print("Origin  Seasonal naive        Zero drift")
for origin, recorded_naive in zip(test_origins, rolling["seasonal_naive"]):
    history = values[:origin]
    naive = seasonal_naive(history, period, horizon)
    zero_drift = seasonal_drift(history, period, horizon, drift=0)
    copied_values = values[origin - period:origin - period + horizon]
    np.testing.assert_array_equal(naive, copied_values)
    np.testing.assert_array_equal(naive, recorded_naive)
    np.testing.assert_array_equal(zero_drift, naive)
    zero_drift_forecasts.append(zero_drift)
    print(f"{origin:>6}  {str(naive.tolist()):<21}  {zero_drift.tolist()}")

targets = np.array(rolling["targets"])
errors = np.abs(targets - np.stack(zero_drift_forecasts))
zero_drift_mae = errors.mean(axis=0)
np.testing.assert_array_equal(zero_drift_mae, [4, 4, 4])
print("Zero-drift MAE", zero_drift_mae.tolist())
```

```
OUTPUT
Origin  Seasonal naive        Zero drift
    24  [20.0, 24.0, 21.0]     [20.0, 24.0, 21.0]
    25  [24.0, 21.0, 25.0]     [24.0, 21.0, 25.0]
    26  [21.0, 25.0, 24.0]     [21.0, 25.0, 24.0]
    27  [25.0, 24.0, 28.0]     [25.0, 24.0, 28.0]
    28  [24.0, 28.0, 25.0]     [24.0, 28.0, 25.0]
    29  [28.0, 25.0, 29.0]     [28.0, 25.0, 29.0]
Zero-drift MAE [4.0, 4.0, 4.0]
```

All six forecasts with zero drift match seasonal naive entry for entry. Both methods have MAE four at every horizon. Only the fitted version adds the change the series makes between seasons. The extra history at later origins updates the values the model copies. The drift used in the original run stays fixed at the value learned from the training prefix.

## 8. Compare interval widths

Imagine issuing the same point forecast with three different margins. The point forecast can be equally accurate in every case, while the wider interval catches an actual value that fell outside the narrower one.

Use the chapter's two-origin fixture. The actual rows are $(10, 12, 15)$ and $(11, 13, 14)$, and the prediction rows are $(10, 11, 14)$ and $(10, 13, 12)$. The first column contains two one-step forecasts, the second two two-step forecasts, and the third two three-step forecasts. Comparing actual values to point predictions gives absolute errors

$$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}.$$

Averaging down the columns gives MAE $(0.5, 0.5, 1.5)$. For half-width $w$, subtract $w$ from each prediction to get the lower endpoint and add $w$ to get the upper one. Each interval then has width $2w$. Coverage at a horizon is the fraction of its two actual values in their respective intervals, including both endpoints. An actual value is covered exactly when its absolute error is at most $w$.

| Half-width | Horizon 1 coverage | Horizon 2 coverage | Horizon 3 coverage | Mean width at every horizon |
|---|---|---|---|---|
| 0 | 1/2 | 1/2 | 0/2 | 0 |
| 1 | 2/2 | 2/2 | 1/2 | 2 |
| 2 | 2/2 | 2/2 | 2/2 | 4 |

The intervals with half-width zero contain only exact predictions. At half-width one, errors of one are covered because the endpoints count. At half-width two, even the largest error is covered. MAE depends only on the actual values and point predictions, so we expect it to stay fixed.

```
from examples.ch27_time_series.model import forecast_metrics

actual = np.array([[10, 12, 15], [11, 13, 14]])
predicted = np.array([[10, 11, 14], [10, 13, 12]])
expected_mae = [0.5, 0.5, 1.5]
expected_coverage = [
    [0.5, 0.5, 0.0],
    [1.0, 1.0, 0.5],
    [1.0, 1.0, 1.0],
]
for half_width in (0, 1, 2):
    lower = predicted - half_width
    upper = predicted + half_width
    mae, coverage, mean_width = forecast_metrics(
        actual, predicted, lower, upper
    )
    np.testing.assert_array_equal(mae, expected_mae)
    np.testing.assert_array_equal(
        coverage, expected_coverage[half_width]
    )
    np.testing.assert_array_equal(mean_width, [2 * half_width] * 3)
    print(f"Half-width {half_width}")
    print("  Lower endpoints", lower.tolist())
    print("  Upper endpoints", upper.tolist())
    print("  MAE             ", mae.tolist())
    print("  Coverage        ", coverage.tolist())
    print("  Mean width      ", mean_width.tolist())
```

OUTPUT

```
Half-width 0
  Lower endpoints [[10, 11, 14], [10, 13, 12]]
  Upper endpoints [[10, 11, 14], [10, 13, 12]]
  MAE              [0.5, 0.5, 1.5]
  Coverage         [0.5, 0.5, 0.0]
  Mean width       [0.0, 0.0, 0.0]
Half-width 1
  Lower endpoints [[9, 10, 13], [9, 12, 11]]
  Upper endpoints [[11, 12, 15], [11, 14, 13]]
  MAE              [0.5, 0.5, 1.5]
  Coverage         [1.0, 1.0, 0.5]
  Mean width       [2.0, 2.0, 2.0]
Half-width 2
  Lower endpoints [[8, 9, 12], [8, 11, 10]]
  Upper endpoints [[12, 13, 16], [12, 15, 14]]
  MAE              [0.5, 0.5, 1.5]
  Coverage         [1.0, 1.0, 1.0]
  Mean width       [4.0, 4.0, 4.0]
```

For the second origin at horizon three, the point prediction is 12 and the actual value is 14. At half-width one, the upper endpoint is 13 and the interval misses. At half-width two, the upper endpoint is 14 and the interval hits. That change brings coverage at horizon three from one hit out of two to two hits out of two. The error for this same point prediction stays at two units. The wider interval buys more coverage at the cost of precision about where the outcome will fall.